PROGRAM 2 — OUTLIER DETECTION USING BOXPLOT, IQR, Z-SCORE

ALGORITHM

Step 1: Import necessary libraries.

Step 2: Load the dataset.

Step 3: Define a function detect_outliers to handle outliers detection for specific column and label.

Step 4: Extract the column values from the dataset using data[column_name].values.

Step 5: Calculate the mean and standard deviation using numpy.mean() and numpy.std(), then compute z-score for all values.

Step 6: Set a z-score threshold (2) for detecting outliers.

Step 7: Using np.percentile(), derive the interquartile range (IQR). Calculate the first and third quartile.

Step 8: Use IQR to calculate the lower and upper bounds for outlier detection (Q1 - 1.5 * IQR and Q3 + 1.5 * IQR).

Step 9: Identify the outliers by comparing z-score and IQR methods.

Step 10: Plot the boxplot for visualizing the distribution using sns.boxplot() and plt.show().

Step 11: Call the function with the specific column and label for analysis.

Step 12: Stop.

PROGRAM 3 — CIFAR-10 CLASSIFICATION

ALGORITHM

Step 1: Import required libraries.

Step 2: Load CIFAR-10 dataset and split into training and testing sets.

Step 3: Normalize image pixel values to range [0,1].

Step 4: Convert class labels into one-hot encoded vectors.

Step 5: Define a sequential model:

- Flatten layer (32, 32, 3)
- Dense (512, ReLU)
- Dense (256, ReLU)
- Dense (128, ReLU)
- Dense (10, softmax)

Step 6: Compile the model with Adam optimizer, categorical-cross-entropy loss and accuracy metric.

Step 7: Train the model with training data for 5 epochs, batch size = 64, with 20% validation split.

Step 8: Evaluate model on test data and record test accuracy.

Step 9: Plot training vs validation accuracy over epochs.

Step 10: Predict on a few test samples and display probability distribution with bar charts.

Step 11: Analyze results and conclude.

Step 12: Stop.

PROGRAM 4 — KAIMING AND XAVIER INITIALIZATION

ALGORITHM

Step 1: Import required libraries.

Step 2: Load CIFAR-10 dataset and split into training and testing sets.

Step 3: Normalize image pixel values to range [0,1].

Step 4: Convert class labels into one-hot-encoded vectors.

Step 5: Define a function to create a sequential model with:

- Flatten layer (32, 32, 3)
- Dense (256, ReLU, initializer, ReLU, L2)
- Dense (128, ReLU, initializer, ReLU, L2)
- Dense (64, ReLU, initializer, ReLU, L2)
- Dense (32, ReLU, initializer, ReLU, L2)
- Dense (10, softmax)

Step 6: Create two models using different weight initializers:

- Xavier initializer (Glorot Normal)
- Kaiming initialization (He Normal)

Step 7: Compile both models with Adam optimizer, categorical cross-entropy loss, and accuracy metric.

Step 8: Train both models for 10 epochs, batch size = 64, with 20% validation split and same history.

Step 9: Plot training vs validation accuracy for both Xavier and Kaiming initialization.

Step 10: Evaluate both models on test data and record test accuracy.

Step 11: Compare results of Xavier vs Kaiming initialization.

Step 12: Analyze results and conclude.

PROGRAM 5 — GRADIENT DESCENT VARIANTS AND ADAPTIVE OPTIMIZATION ALGORITHMS

ALGORITHM

Step 1: Import necessary libraries.

Step 2: Load MNIST dataset and split into training and testing sets.

Step 3: Normalize image pixel values to range [0,1] (255.0).

Step 4: Reshape images to include channel dimensions.

Step 5: Convert class labels into one-hot encoded vectors.

Step 6: Define a create_model() function returning a Sequential CNN.

Step 7: Define the optimizer dictionary to compare Stochastic Gradient Descent, SGD with Momentum, SGD + Nesterov, Adam, RMSProp, Adagrad.

Step 8: For each optimizer:

Step 9: Instantiate a fresh model via create_model().

Step 10: Compile with that optimizer, loss='categorical_crossentropy', metrics=['accuracy'].

Step 11: Train with fit(train_images, train_labels, epochs=5, batch_size=64, validation_split=0.2, verbose=1).

Step 12: Save training history and use the trained model for evaluation.

Step 13: Plot comparisons for validation accuracy and validation loss.

Step 14: Evaluate each saved model on the test set and record the final test accuracy.

Step 15: Analyze and conclude.

PROGRAM 6 — MNIST DIGIT CLASSIFICATION USING CNN ARCHITECTURE

ALGORITHM

Step 1: Import necessary libraries.

Step 2: Load the MNIST dataset.

Step 3: Normalize images to scale pixel values to range [0,1].

Step 4: Reshape images and add channels to match CNN input shape.

Step 5: Convert labels into one-hot encoded vectors using to_categorical().

Step 6: Build a Sequential model with layers:

- Conv2D (32, 3×3, ReLU)
- MaxPooling2D (2×2)
- Conv2D (64, 3×3, ReLU)
- MaxPooling2D (2×2)
- Conv2D (64, 3×3, ReLU)
- Flatten layer
- Dense layer with 64 units and ReLU activation
- Dense layer with 10 units and softmax activation

Step 7: Compile the model using Adam optimizer, categorical cross-entropy loss and accuracy metric.

Step 8: Train the model for 5 epochs with a batch size of 64 and a validation split of 0.2.

Step 9: Evaluate the trained model on the test data and record the test accuracy.

Step 10: Plot graphs for training vs validation accuracy and training vs validation loss.

Step 11: Make predictions for the first 10 test images.

Step 12: Display predictions and compare with actual labels.

Step 13: Stop.

LAST PROGRAM — SENTIMENT ANALYSIS USING PRE-TRAINED BERT

ALGORITHM

Step 1: START.

Step 2: Install the required libraries: transformers and torch to use pre-trained NLP models.

Step 3: Import the pipeline function from the transformers library.

Step 4: Define a function analyze_sentiment(text) to analyze sentiment of the given text.

Step 5: Initialize the sentiment-analysis pipeline using the pre-trained model.

Step 6: Use the pipeline to predict the sentiment label and confidence score.

Step 7: Define three simple input sentences: a positive sentence, a negative sentence, and a neutral sentence.

Step 8: Call the analyze_sentiment() function for each example sentence.

Step 9: Display the original sentence, predicted sentiment label, and confidence score.

Step 10: Stop.

PROGRAM 7 — DIGIT CLASSIFICATION ON MNIST USING VGG-19

AIM

To implement a Python program for digit classification using pre-trained networks like VGGNet-19, AlexNet, etc. for MNIST dataset and analyze and visualize performance improvement.

ALGORITHM

Step 1: Import required libraries.

Step 2: Load the MNIST dataset and split it into training and testing sets.

Step 3: Convert class labels into one-hot encoded vectors using to_categorical().

Step 4: Expand dimensions of image to (28, 28, 1). Resize all images to (48, 48) using smart_resize. Convert grayscale images to 3-channel RGB using np.repeat. Normalize pixel values to range [0, 1].

Step 5: Import VGG19 model with imagenet weights. Exclude the fully connected top layers. Set input shape to (48, 48, 3). Freeze all layers of VGG19.

Step 6: Build a sequential model with the following structure:

- VGG-19 (pretrained, frozen)
- Flatten layer
- Dense layer with 128 units and ReLU activation.
- Dropout layer with rate 0.5.
- Dense output layer with 10 units and softmax activation.

Step 7: Compile the model using Adam optimizer, categorical cross entropy loss function, and accuracy as evaluation metrics.

Step 8: Train the model for 10 epochs with a batch size of 64 using the training set and validate using the test set.

Step 9: Evaluate the trained model on the test data and record the test accuracy.

Step 10: Plot graphs for training vs validation accuracy and training vs validation loss across epochs.

Step 11: Analyze and conclude.

PROGRAM 10 — TIME SERIES FORECASTING PREDICTION

AIM

To implement time series forecasting prediction for NIFTY-50 dataset.

ALGORITHM

Step 1: START.

Step 2: Import the necessary libraries.

Step 3: Download the NIFTY-50 Index from YahooFinance and keep only the close price column.

Step 4: Normalize the data by applying Min-Max scaling, converting all prices to a range between 0 and 1.

Step 5: Create input and output sequences where each input consists of the previous 60 days' prices and the output is the next day's price.

Step 6: Reshape the input data into 3D format suitable for LSTM.

Step 7: Build the LSTM Model:

Step 7.1: Add an LSTM layer with 50 units and return sequences.

Step 7.2: Add a Dropout layer to reduce overfitting.

Step 7.3: Add another LSTM layer with 50 units.

Step 7.4: Add another Dropout layer.

Step 7.5: Add a Dense layer with one output unit.

Step 8: Compile the model using the Adam optimizer and Mean Squared Error (MSE) as the loss function.

Step 9: Train the model on the prepared data for 20 epochs with a batch size of 32.

Step 10: Use the trained model to predict the next day's closing price on the last 60 days of input data.

Step 11: Inverse transform the predicted value to obtain the actual price in rupees.

Step 12: Plot the graph for visual comparison by showing the actual vs predicted stock prices.

Step 13: Display the predicted next day closing price.

Step 14: Stop.